

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **HEMASRINITHA P. (192572179)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty**
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic**
- iii. Analyze the impact of heuristic accuracy on both algorithms**
- iv. Discuss how dynamic changes (blocked paths) affect search performance**
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety**

Given Scenario: A government uses AI-powered drones to deliver emergency medicines in flood-affected areas. The drone must find the best route despite blocked roads, changing weather, and different travel costs.

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic (straight-line distance). It does not consider the actual travel cost.

Strengths:

- Finds a route quickly.
- Requires less computation.
- Suitable when fast decisions are needed.

Weaknesses:

- Ignores movement costs like weather and terrain.
- May choose blocked or unsafe paths.

- Does not always find the optimal route.

In this scenario, GBFS can respond quickly but may fail if the chosen path becomes inaccessible.

ii. A* Search

A* Search uses both the actual cost from the start ($g(n)$) and the estimated cost to the goal ($h(n)$).

Formula:

$$f(n) = g(n) + h(n)$$

This helps the drone choose a path that is both short and safe by considering travel cost and distance.

Advantages:

- Finds the optimal path when the heuristic is accurate.
- Avoids costly or risky routes.
- More reliable than Greedy Search in dynamic environments.

iii. Impact of Heuristic Accuracy

The heuristic estimates the remaining distance to the destination.

- Accurate heuristic: Both GBFS and A* perform efficiently and explore fewer nodes.
- Inaccurate heuristic: GBFS may follow wrong paths because it depends only on the heuristic, while A* still considers the actual cost and usually finds a better route.

Thus, A* is less affected by heuristic errors than GBFS.

iv. Effect of Dynamic Changes (Blocked Paths)

When roads become blocked during the mission:

- GBFS: May continue toward the blocked path and may need to restart the search.
- A*: Can update the route by considering the new costs and blocked paths, making it more adaptable.

Dynamic changes reduce the performance of both algorithms, but A* handles them more effectively.

v. Recommended Algorithm

Recommended: A* Search

Justification:

- Considers both travel cost and estimated distance.
- Finds safer and more efficient routes.
- Adapts better to changing conditions.
- Provides a good balance between speed, optimality, and safety.

Although Greedy Best-First Search is faster, A* is the better choice for emergency medicine delivery because it ensures reliable and safe navigation in a dynamic environment.

Conclusion

For emergency drone navigation, A* is more suitable than Greedy Best-First Search because it considers both actual travel cost and heuristic information. This allows the drone to avoid risky routes and reach the destination safely with minimum delay, making it ideal for real-time emergency situations.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Given Scenario: A smart city uses an AI system to optimize traffic signal timings at hundreds of intersections. The goal is to reduce waiting time, fuel consumption, and traffic congestion. Since traffic conditions change continuously, the search space is very large and finding the best solution is challenging.

Problem Formulation

This problem can be treated as an optimization problem, where the objective is to find the best traffic signal timings that minimize waiting time, fuel consumption, and congestion.

Traditional search algorithms are inefficient because:

- The number of possible signal timing combinations is extremely large.
- Traffic conditions change frequently.
- Searching every possible solution takes too much time.

i. Hill Climbing

Hill Climbing starts with an initial traffic signal configuration and continuously moves to a neighboring solution that improves traffic flow.

Advantages:

- Simple and easy to implement.
- Produces quick improvements.
- Suitable for small optimization problems.

Limitations:

- Can get stuck in a local optimum.
- Cannot guarantee the best overall solution.
- May stop even when better solutions exist.

ii. Local Maxima, Plateaus, and Ridges

Local Maxima: The algorithm reaches a solution that is better than nearby solutions but not the best overall.

Example: One junction has smooth traffic, but nearby intersections remain congested.

Plateau: Many neighboring solutions have the same performance, so the algorithm cannot decide which direction to move.

Example: Changing signal timings by a few seconds shows no improvement in traffic flow.

Ridge: The best solution requires several coordinated changes, but the algorithm changes only one signal at a time.

Example: Traffic improves only when multiple intersections are adjusted together.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Occasionally accepts worse solutions to escape local maxima.
- Gradually improves the solution as the search progresses.

Genetic Algorithm

- Uses a population of solutions instead of one solution.
- Applies selection, crossover, and mutation to generate better solutions.
- Explores multiple possibilities and avoids getting stuck in local optima.

Both methods perform better than Hill Climbing in large and dynamic search spaces.

iv. Recommended Approach

Recommended: Genetic Algorithm

Justification:

- Handles large search spaces efficiently.
- Adapts to changing traffic conditions.

- Produces high-quality solutions.
- Suitable for real-time smart city traffic management.

Conclusion

For traffic signal optimization, Genetic Algorithms are the most suitable approach because they efficiently search large solution spaces, adapt to changing traffic conditions, and provide better optimization than Hill Climbing. This makes them ideal for improving traffic flow in a smart city.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- Explain why this problem requires an online search agent instead of offline search**
- Analyze the challenges of partial observability and dynamic environments**
- Describe how the rover builds and updates its internal model**
- Compare online search with uninformed and informed search strategies**
- Justify the most suitable approach considering uncertainty, adaptability, and safety.**

Given Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must collect samples, avoid hazards such as rocks and craters, operate with limited sensing capability, and make decisions without a complete map. As it explores, it continuously updates its knowledge of the environment.

i. Online and offline search agent

An **Online Search Agent** is suitable because the rover does not have complete knowledge of the environment before starting. It must explore the terrain, gather information, and make

decisions while moving. An **Offline Search** requires a complete map before planning a path, which is not possible in an unknown Martian environment.

ii. Challenges of Partial Observability and Dynamic Environments

Partial Observability

- The rover has limited sensor range.
- It cannot detect all obstacles in advance.
- It must make decisions with incomplete information.

Dynamic Environment

- New hazards may be discovered during exploration.
- Dust storms or shifting terrain can change the environment.
- The rover must continuously adapt to these changes.

iii. How does the Rover Build and Update its Internal Model?

The rover creates an internal map as it explores by:

- Recording visited locations.
- Identifying safe and unsafe paths.
- Marking obstacles such as rocks and craters.
- Updating its knowledge using sensor data.

This updated model helps the rover make better decisions during future movements.

iv. Compare Online Search with Uninformed and Informed Search

Online Search	Uninformed Search	Informed Search
Learns while exploring	Requires a known environment	Uses heuristic information

Online Search	Uninformed Search	Informed Search
Adapts to new obstacles	Does not adapt easily	Performs better when a map is available
Suitable for unknown environments	Suitable for simple known problems	Suitable for known environments with heuristics

v. Recommended Approach

Recommended: Online Search Agent

Justification:

- Works effectively in unknown environments.
- Adapts to newly discovered obstacles.
- Continuously updates its knowledge.
- Supports safe and efficient navigation.
- Suitable for real-time decision making.

Conclusion

For Mars exploration, an Online Search Agent is the best approach because it allows the rover to learn about the environment while exploring. It can adapt to uncertainty, avoid hazards, and make safe decisions in real time, making it more effective than traditional offline search methods.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks : Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation**
- ii. Explain how backtracking search works in this scenario**
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency**
- iv. Analyze real-world challenges and justify the best strategy for scalability.**

Given Scenario: A university wants to develop an AI system to automatically generate exam timetables. The system must ensure that there are no exam clashes, exam halls are available, faculty members are free, subject order is maintained, and special accommodations are provided for certain students.

i. Variables, Domains, and Constraints

Variables: Exams to be scheduled.

Domains: Available dates, Time slots, Exam halls.

Constraints

- No student should have overlapping exams.
- Exam halls should not exceed their capacity.
- Faculty must be available during the exam.
- Certain subjects must be scheduled before others.
- Students requiring special accommodations must be assigned suitable halls.

These constraints ensure that the timetable is valid and conflict-free.

ii. Backtracking Search

Backtracking Search assigns one exam at a time to a date, time slot, and hall.

- If an assignment satisfies all constraints, it proceeds to the next exam.
- If a conflict occurs, it goes back (backtracks) and tries another available option.

- This process continues until a complete timetable is generated.

Backtracking guarantees a valid solution if one exists.

iii. Constraint Propagation (Forward Checking)

Forward Checking improves backtracking by removing invalid choices after each assignment.

For example, if an exam is scheduled on Monday morning, that time slot is removed for other exams involving the same students or faculty.

Benefits:

- Detects conflicts early.
- Reduces the search space.
- Speeds up timetable generation.
- Avoids unnecessary backtracking.

iv. Real-World Challenges and Best Strategy

Challenges

- Large number of students and courses.
- Limited exam halls.
- Faculty availability changes.
- Last-minute modifications.
- Increasing complexity as the university grows.

Recommended Strategy: Backtracking Search with Forward Checking

Justification:

- Efficiently handles multiple constraints.
- Reduces unnecessary searches.

- Produces conflict-free timetables.
- Scales better for large universities.

Conclusion

The exam scheduling problem is best modeled as a Constraint Satisfaction Problem (CSP). Using Backtracking Search along with Forward Checking helps generate an efficient and conflict-free timetable while satisfying all academic and resource constraints. This approach is scalable and suitable for real-world university scheduling systems.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem**
- ii. Analyze the computational challenges of large game trees**
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**
- iv. Design an evaluation function and justify the factors considered**
- v. Recommend a strategy for balancing optimality and real-time performance**

Given Scenario: A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within limited time, evaluate many possible moves, and handle a very large decision tree efficiently.

i. Explain how the Minimax algorithm models this problem

The **Minimax algorithm** models the game as a two-player decision-making problem. It assumes that:

- The AI (Max player) tries to maximize its score.
- The human opponent (Min player) tries to minimize the AI's score.

The algorithm explores possible future moves and selects the move that gives the AI the best possible outcome, assuming the opponent also plays optimally.

ii. Analyze the computational challenges of large game trees

In real-time strategy games, each move creates many possible future moves, resulting in a very large game tree.

Challenges:

- Large number of possible game states.
- High computation time.
- Increased memory usage.
- Difficult to make decisions within strict time limits.

Searching the entire game tree becomes impractical.

iii. Explain how Alpha-Beta Pruning improves efficiency

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot influence the final decision.

Instead of exploring every possible move, it skips unnecessary branches and focuses only on promising ones.

Advantages:

- Reduces the number of nodes explored.
- Speeds up decision making.
- Produces the same optimal result as Minimax.
- Allows deeper search within the available time.

iv. Design an evaluation function and justify the factors considered

The evaluation function estimates the strength of a game position when the search cannot reach the end of the game.

It may consider factors such as:

- Number of remaining units.
- Health of units.
- Resources collected.
- Territory controlled.
- Distance to objectives.
- Defensive and attacking strength.

These factors help the AI choose the most beneficial move.

v. Recommend a strategy for balancing optimality and real-time performance

Recommended Strategy: Minimax with Alpha-Beta Pruning

Justification:

- Makes intelligent decisions quickly.
- Reduces unnecessary computations.
- Maintains the quality of decisions.
- Suitable for real-time games with limited decision time.
- Provides a good balance between optimality and performance.

Conclusion

For real-time strategy games, Minimax with Alpha-Beta Pruning is the most suitable approach. It enables the AI to make strong decisions while reducing computation time, making it efficient enough for real-time gameplay without sacrificing decision quality.

